DATA STRUCTURES (LAB)

LAB TASK 07 and 08



By

MISBAH KHALID

FA25-BCS-005

Submitted to,

Mr. BILAL BUKHARI

October 22, 2026

Department of Computer Sciences

COMSATS University Islamabad,

Attock Campus

LAB 07

TASK 01

CODE:

```
#include <iostream>
using namespace std;

// Structure for a tree node
struct Node {
   int data;          // Holds the node's value
   Node* left;        // Pointer to left child
   Node* right;       // Pointer to right child

   // Constructor to initialize a new node
   Node(int val) {
      data = val;     // Set the node's data
      left = nullptr; // Initialize left child as null
      right = nullptr;// Initialize right child as null
   }
};

// 1. Pre Order Traversal (NRL - Node, Right, Left)
void preOrderNRL(Node* root) {
   if (root == nullptr) return; // Base case: stop if current node is null
   cout << root->data << " ";   // Process Node first (N)
   preOrderNRL(root->right);    // Traverse Right subtree second (R)
   preOrderNRL(root->left);     // Traverse Left subtree last (L)
}

// 2. Post Order Traversal (RLN - Right, Left, Node)
void postOrderRLN(Node* root) {
   if (root == nullptr) return; // Base case: stop if current node is null
   postOrderRLN(root->right);   // Traverse Right subtree first (R)
   postOrderRLN(root->left);    // Traverse Left subtree second (L)
   cout << root->data << " ";   // Process Node last (N)
}

// 3. In Order Traversal (RNL - Right, Node, Left)
void inOrderRNL(Node* root) {
   if (root == nullptr) return; // Base case: stop if current node is null
   inOrderRNL(root->right);     // Traverse Right subtree first (R)
   cout << root->data << " ";   // Process Node second (N)
   inOrderRNL(root->left);      // Traverse Left subtree last (L)
```

```
}

int main() {
    // Constructing a sample tree:
    //        10
    //       /  \
    //      5    15
    //     / \   / \
    //    3   7 12  20
    Node* root = new Node(10);
    root->left = new Node(5);
    root->right = new Node(15);
    root->left->left = new Node(3);
    root->left->right = new Node(7);
    root->right->left = new Node(12);
    root->right->right = new Node(20);

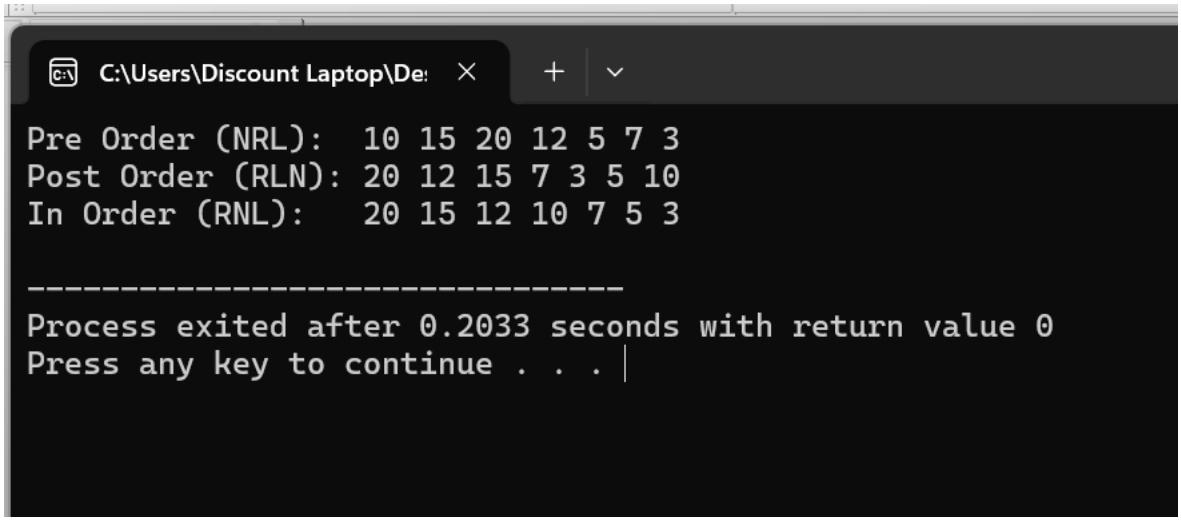
    cout << "Pre Order (NRL):  ";
    preOrderNRL(root); // Call NRL traversal
    cout << endl;

    cout << "Post Order (RLN): ";
    postOrderRLN(root); // Call RLN traversal
    cout << endl;

    cout << "In Order (RNL):   ";
    inOrderRNL(root); // Call RNL traversal
    cout << endl;

    return 0;
}
```

OUTPUT:



TASK 02

CODE:

```
#include <iostream>
using namespace std;

// Structure for a tree node
struct Node {
    int data;          // Holds the node's value
    Node* left;        // Pointer to left child
    Node* right;       // Pointer to right child

    // Constructor to initialize a new node
    Node(int val) {
        data = val;    // Set the node's data
        left = nullptr; // Initialize left child as null
        right = nullptr;// Initialize right child as null
    }
};

// Function to print and count leaf nodes
int printAndCountLeaves(Node* root) {
    // Base case: if the tree is empty, there are 0 leaves
    if (root == nullptr) {
        return 0;
    }

    // Check if the current node is a leaf node
    // (It has no left child AND no right child)
    if (root->left == nullptr && root->right == nullptr) {
        cout << root->data << " "; // Print the leaf node's data
```

```
        return 1;                // Return 1 to add to the total count
    }

    // If it's not a leaf, recursively check both left and right subtrees
    // and add their counts together
    int leftLeaves = printAndCountLeaves(root->left);   // Count leaves in left
subtree
    int rightLeaves = printAndCountLeaves(root->right); // Count leaves in right
subtree

    return leftLeaves + rightLeaves; // Return the total count for this branch
}

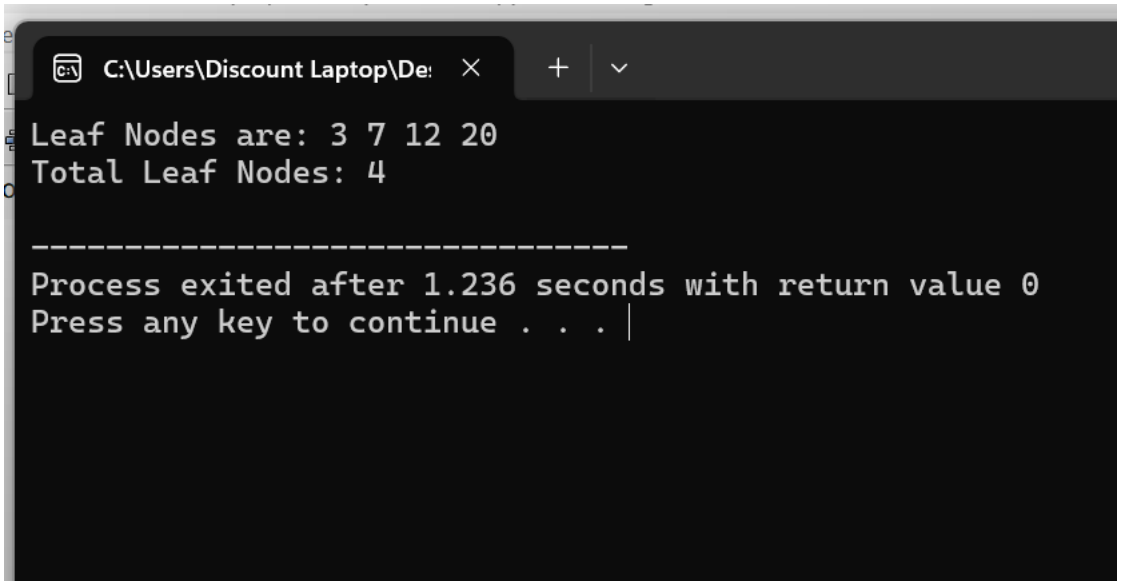
int main() {
    // Constructing a sample Binary Search Tree:
    //        10
    //       /  \
    //      5    15
    //     / \   / \
    //    3   7 12  20
    Node* root = new Node(10);
    root->left = new Node(5);
    root->right = new Node(15);
    root->left->left = new Node(3);
    root->left->right = new Node(7);
    root->right->left = new Node(12);
    root->right->right = new Node(20);

    cout << "Leaf Nodes are: ";
    int totalLeaves = printAndCountLeaves(root); // Call the function

    cout << "\nTotal Leaf Nodes: " << totalLeaves << endl; // Print final count

    return 0;
}
```

OUTPUT:



TASK 03
CODE

```
#include <iostream>
using namespace std;

// Structure for a tree node
struct Node {
    int data;          // Holds the node's value
    Node* left;        // Pointer to left child
    Node* right;       // Pointer to right child

    // Constructor to initialize a new node
    Node(int val) {
        data = val;    // Set the node's data
        left = nullptr; // Initialize left child as null
        right = nullptr;// Initialize right child as null
    }
};

// Helper function to find the node with the minimum value in a tree
// Used in Case 3 (Node with two children) to find the Inorder Successor
Node* findMin(Node* root) {
    while (root->left != nullptr) { // Keep traversing left until we can't anymore
        root = root->left;         // Move pointer to the left child
    }
    return root;                   // Return the leftmost (smallest) node
}
```

```
// Function to delete a node from the BST
Node* deleteNode(Node* root, int key) {
    // Base case: if the tree is empty, return null
    if (root == nullptr) {
        return root;
    }

    // Step 1: Search for the node to delete
    if (key < root->data) {
        // Key is smaller, go to left subtree
        root->left = deleteNode(root->left, key);
    }
    else if (key > root->data) {
        // Key is larger, go to right subtree
        root->right = deleteNode(root->right, key);
    }
    else {
        // Step 2: Node found! Handle the 3 possibilities for deletion:

        // Possibility 1 & 2: Node with NO children (leaf) OR node with ONE
child
        if (root->left == nullptr) {
            Node* temp = root->right; // Store right child (or null if leaf)
            delete root;              // Delete the current node
            return temp;              // Return right child to replace current node
        }
        else if (root->right == nullptr) {
            Node* temp = root->left;  // Store left child
            delete root;              // Delete the current node
            return temp;              // Return left child to replace current node
        }

        // Possibility 3: Node with TWO children
        // Find the Inorder Successor (smallest node in the right subtree)
        Node* temp = findMin(root->right);

        // Copy the successor's data to the current node
        root->data = temp->data;

        // Delete the successor node from the right subtree
        root->right = deleteNode(root->right, temp->data);
    }
```

```
    return root; // Return the modified tree root
}

// Helper function to print tree in standard Inorder (Left, Node, Right) to verify
deletions
void printInOrder(Node* root) {
    if (root == nullptr) return;
    printInOrder(root->left);
    cout << root->data << " ";
    printInOrder(root->right);
}

int main() {
    // Constructing a sample BST:
    //        50
    //       /  \
    //     30    70
    //    / \  / \
    //  20  40 60  80
    Node* root = new Node(50);
    root->left = new Node(30);
    root->right = new Node(70);
    root->left->left = new Node(20);
    root->left->right = new Node(40);
    root->right->left = new Node(60);
    root->right->right = new Node(80);

    cout << "Original Tree (In-Order): ";
    printInOrder(root);
    cout << endl << endl;

    // --- TEST CASE 1: Delete a Leaf Node (no children) ---
    cout << "Deleting 20 (Leaf node)..." << endl;
    root = deleteNode(root, 20);
    cout << "Tree after deleting 20: ";
    printInOrder(root);
    cout << endl << endl;

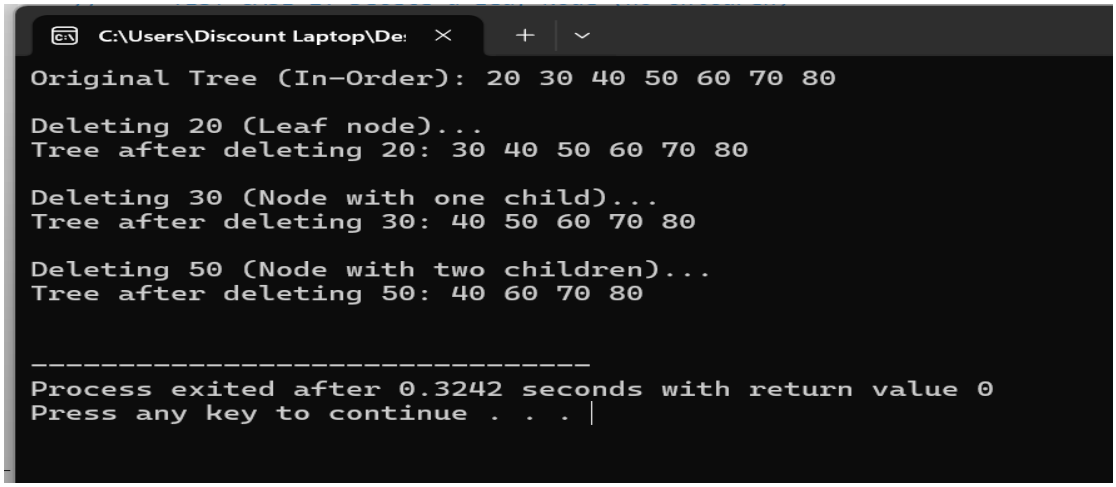
    // --- TEST CASE 2: Delete a Node with One Child ---
    // (Let's delete 30, it now only has a right child 40)
    cout << "Deleting 30 (Node with one child)..." << endl;
    root = deleteNode(root, 30);
    cout << "Tree after deleting 30: ";
```

```
    printInOrder(root);
    cout << endl << endl;

    // --- TEST CASE 3: Delete a Node with Two Children ---
    // (Deleting 50, the root, which has two children)
    cout << "Deleting 50 (Node with two children)..." << endl;
    root = deleteNode(root, 50);
    cout << "Tree after deleting 50: ";
    printInOrder(root);
    cout << endl << endl;

    return 0;
}
```

OUTPUT:



LAB 08:

TASK 01

CODE

```
#include <iostream>
#include <algorithm> // Required for the max() function
using namespace std;

// Structure for an AVL Tree Node
struct Node {
    int data;           // Stores the data value
    Node* left;         // Pointer to the left child
    Node* right;        // Pointer to the right child
    int height;         // Stores the height of the node
```

```
    // Constructor to initialize a new node
    Node(int val) {
        data = val;     // Set the data
        left = nullptr; // Initialize left child as null
        right = nullptr;// Initialize right child as null
        height = 1;     // A new node is initially at height 1
    }
};

// Utility function to get the height of a node
int getHeight(Node* node) {
    if (node == nullptr) return 0; // If node is null, height is 0
    return node->height;           // Otherwise, return the stored height
}

// Utility function to calculate the Balance Factor
// Balance Factor = height(left subtree) - height(right subtree)
int getBalance(Node* node) {
    if (node == nullptr) return 0; // If node is null, balance is 0
    return getHeight(node->left) - getHeight(node->right); // Calculate and return
balance
}

// --- ROTATION 1: Right Rotation (LL Case) ---
Node* rotateRight(Node* y) {
    Node* x = y->left;      // x is the left child of y
    Node* T2 = x->right;    // T2 is the right child of x (which will move)

    // Perform rotation
    x->right = y;           // x becomes the new root, y becomes its right child
    y->left = T2;           // T2 becomes the left child of y

    // Update heights (must update y first, as it is now lower)
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;

    return x;               // Return the new root (x)
}

// --- ROTATION 2: Left Rotation (RR Case) ---
Node* rotateLeft(Node* x) {
    Node* y = x->right;     // y is the right child of x
```

```
    Node* T2 = y->left;     // T2 is the left child of y (which will move)

    // Perform rotation
    y->left = x;            // y becomes the new root, x becomes its left child
    x->right = T2;          // T2 becomes the right child of x

    // Update heights (must update x first, as it is now lower)
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;

    return y;               // Return the new root (y)
}

// Function to insert a node and automatically apply necessary rotations
Node* insert(Node* node, int key) {
    // 1. Perform standard BST insertion
    if (node == nullptr) return new Node(key); // Create new node if empty

    if (key < node->data) {
        node->left = insert(node->left, key);  // Insert in left subtree
    } else if (key > node->data) {
        node->right = insert(node->right, key);// Insert in right subtree
    } else {
        return node; // Duplicate keys not allowed, return unchanged
    }

    // 2. Update height of this ancestor node
    node->height = 1 + max(getHeight(node->left), getHeight(node->right));

    // 3. Get the balance factor to check if this node became unbalanced
    int balance = getBalance(node);

    // 4. If unbalanced, apply rotations (4 Cases)

    // Case 1: Left Left (LL) -> Apply Right Rotation
    if (balance > 1 && key < node->left->data) {
        return rotateRight(node);
    }

    // Case 2: Right Right (RR) -> Apply Left Rotation
    if (balance < -1 && key > node->right->data) {
        return rotateLeft(node);
    }
```

```
    // Case 3: Left Right (LR) -> Apply Left on child, then Right on node
    if (balance > 1 && key > node->left->data) {
        node->left = rotateLeft(node->left); // First, rotate left on left child
        return rotateRight(node);           // Then, rotate right on current node
    }

    // Case 4: Right Left (RL) -> Apply Right on child, then Left on node
    if (balance < -1 && key < node->right->data) {
        node->right = rotateRight(node->right); // First, rotate right on right child
        return rotateLeft(node);               // Then, rotate left on current node
    }

    return node; // Return the (potentially new) root of this subtree
}

// Helper function to print the tree in Pre-order (Root, Left, Right)
void printPreOrder(Node* root) {
    if (root == nullptr) return;       // Base case
    cout << root->data << " ";         // Print node
    printPreOrder(root->left);         // Go left
    printPreOrder(root->right);        // Go right
}

int main() {
    Node* root = nullptr; // Initialize empty tree

    // Inserting nodes. The AVL insert function will automatically
    // figure out which rotations to apply to keep the tree balanced.
    root = insert(root, 10);
    root = insert(root, 20);
    root = insert(root, 30); // Triggers RR Rotation (Left Rotation)

    root = insert(root, 40);
    root = insert(root, 50);

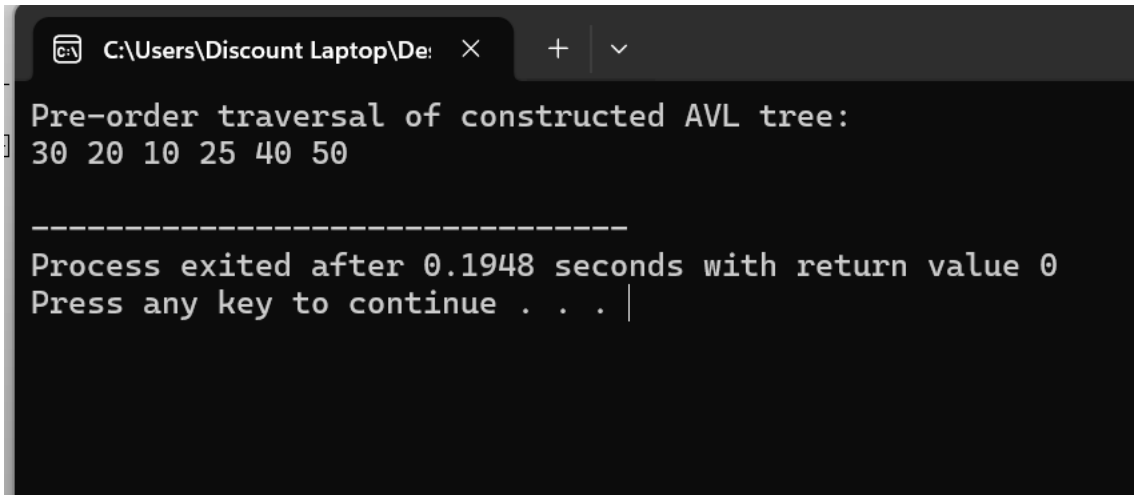
    root = insert(root, 25); // Triggers RL Rotation (Right-Left Rotation)

    cout << "Pre-order traversal of constructed AVL tree:\n";
    printPreOrder(root);
    cout << endl;

    return 0;
```

}

OUTPUT:



TASK 02
CODE:

```
#include <iostream>
#include <queue> // Required for level order traversal
using namespace std;

// Structure for a Tree Node
struct Node {
    int data;          // Stores the data value
    Node* left;        // Pointer to left child
    Node* right;       // Pointer to right child

    // Constructor to initialize a new node
    Node(int val) {
        data = val;     // Set data
        left = nullptr; // Set left to null
        right = nullptr;// Set right to null
    }
};

// Function to perform Level Order Traversal
void printLevelOrder(Node* root) {
    // Base case: if the tree is empty, do nothing
    if (root == nullptr) return;
```

```
    // Create a queue to hold nodes for processing
    queue<Node*> q;

    // Push the root node into the queue to start
    q.push(root);

    // Continue processing while there are nodes in the queue
    while (!q.empty()) {
        // Get the front node from the queue
        Node* current = q.front();

        // Remove the processed node from the queue
        q.pop();

        // Print the data of the current node
        cout << current->data << " ";

        // If the left child exists, push it to the queue for later processing
        if (current->left != nullptr) {
            q.push(current->left);
        }

        // If the right child exists, push it to the queue for later processing
        if (current->right != nullptr) {
            q.push(current->right);
        }
    }
}

int main() {
    // Manually constructing a balanced tree to represent an AVL tree
    //        10
    //       /  \
    //      5    15
    //     / \   / \
    //    3   7 12  20
    Node* root = new Node(10);
    root->left = new Node(5);
    root->right = new Node(15);
    root->left->left = new Node(3);
    root->left->right = new Node(7);
    root->right->left = new Node(12);
    root->right->right = new Node(20);
```

```
    cout << "Level Order Traversal of AVL Tree: ";
    printLevelOrder(root); // Call the function
    cout << endl;

    return 0;
}
```

OUTPUT:

